

Week 5 — Lab Handout (Student)

Controllers, Tests & the Checkpoint

Time: ~90 min in-class (includes the individual checkpoint) **Work as:** Your team (5 members) — except the checkpoint, which is solo **AI use this week: X OFF.** Final week of the Fundamentals Wall. **Due at end of session:** Deliverable 2 (Routing, Logic & Tests, 25%)

Why this lab matters

Today you complete the logic layer — the controllers that actually do the work — and prove it works with automated tests. Then each of you demonstrates, solo, that you can build a route yourself. This is the payoff of three AI-off weeks: real, defensible skill you'll rely on for the rest of the course.

Task 1 — Implement thin controllers (25 min)

For each CRUD operation, write a controller that takes **validated** data, calls the data layer, and returns a **standardized response**. Keep it thin. Pseudocode:

```
function createOrder(request):
  data = request.validatedBody      # validation already passed (Week 4)
  order = orders.save(data)         # data layer does the work
  return response(201, { data: order })
```

- ☐ A controller exists for each CRUD operation on each record type.
- ☐ The route → validation → controller pipeline is wired end to end.
- ☐ Controllers are thin: no validation or raw persistence stuffed inside.

Task 2 — Standardize your success response (10 min)

Make success responses mirror the error shape from Week 4 — one envelope everywhere.

```
success: { "status": 201, "data": { ...record... } }
error:   { "status": 422, "error": "qty out of range", "field": "qty" }
```

- ☐ Every controller returns the same success shape.
- ☐ Success and error envelopes are consistent across the whole team.

Task 3 — Write automated tests (25 min)

For each controller, write tests using **Arrange-Act-Assert**. Cover three kinds of case:

```
test "createOrder saves a valid order":      # happy path
  request = fakeRequest(body = { item: "Cake", qty: 2 })
  result = createOrder(request)
  expect(result.status == 201)

test "createOrder rejects negative quantity": # validation failure
  request = fakeRequest(body = { item: "Cake", qty: -1 })
  result = createOrder(request)
  expect(result.status == 422)
```

- ☐ Each controller has at least: one happy-path test, one validation-failure test, one edge case.
- ☐ The full test suite **runs and passes** (green).
- ☐ Tests assert something meaningful — no `expect(true)` filler.

Merge rule: no code merges to `main` with a failing test. A red suite blocks the PR.

Task 4 — The AI-off checkpoint (individual, ~20 min)

Each team member completes this **alone, unassisted, no AI**, while the instructor observes. You'll get one small task — build or fix a single route end to end (route + validation + controller + one test) against a short spec.

- ☐ Every team member completes their checkpoint task.
- ☐ Work is done solo — no teammate help, no AI, no copy-paste.

This is part of your **individual** grade for Deliverable 2. It's also how the instructor spots who needs a hand before AI turns on next week — treat it as a check-up, not a trap.

Task 5 — Assemble Deliverable 2 (10 min)

Confirm the full submission (see the Deliverable 2 spec for the complete Definition of Done):

- ☐ Consistent routing structure (`/docs/routes.md`) — from Week 3
- ☐ Validation on every create/update route (`/docs/validation.md`) — from Week 4
- ☐ Thin controllers completing core CRUD, wired end to end
- ☐ A passing automated test suite (happy path + failures + edges)
- ☐ Every member owns controllers/tests on the board; all merges via reviewed PR
- ☐ Each member's checkpoint completed

End-of-Lab Checklist

- ☐ Controllers done and wired; responses standardized
- ☐ Test suite green, with meaningful assertions
- ☐ Individual checkpoints completed
- ☐ Deliverable 2 submitted; contributions visible on the board
- ☐ No AI used this phase — confirm honestly

Looking ahead to Phase 3 (Week 6)

AI turns ON. With your logic layer built and tested by hand, you'll start binding the interface — and begin using AI as a tool, with prompt logs required. Everything you built these three weeks is what lets you judge whether AI's output is actually correct.